How-To Library

Informatica

Tuning PowerExchange CDC Publisher to Improve Performance

© Copyright Informatica LLC 2020, 2021. Informatica, the Informatica logo, PowerExchange, and PowerCenter are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

This article describes how to tune PowerExchange CDC Publisher for improved performance.

Supported Versions

- PowerExchange 1.3

Table of Contents

Overview. 2
Disabling CDC Publisher Guaranteed Delivery. 2
Enabling CDC Publisher Asynchronous Communication with Kafka. 3
CDC Publisher Leveraging of Kafka Partitioning. 4
Reducing Checkpointing I/O. 4
Increasing the Size of the CDC Publisher Internal Pending Buffer 5
Increasing the Size of Selected CDC Publisher Internal Queues. 7
Hardware. 7
Monitoring Performance. 7

Overview

To tune PowerExchange CDC Publisher to meet your performance requirements, use one or more of the following methods:

- Disable CDC Publisher guaranteed delivery
- Enable CDC Publisher asynchronous communication with Kafka
- Leverage Kafka partitioning
- Reduce checkpointing I/O
- Increase the size of the CDC Publisher internal pending buffer
- Increase the size of selected CDC Publisher internal queues
- Hardware
- Monitor performance

Note: This article uses the term "message" to refer to an object that is sent to Kafka.

Disabling CDC Publisher Guaranteed Delivery

Guaranteed delivery is enabled by default in CDC Publisher. With guaranteed delivery enabled, CDC Publisher attempts to deliver messages to Kafka exactly once. However, this effort has a high performance cost because messages are sent synchronously, one at a time, to the target. Conversely, asynchronous communication with Kafka provides for

better CDC Publisher performance by reducing the number of duplicate messages sent and by allowing messages to be sent to Kafka before an acknowledge of message receipt is received.

Managing CDC Publisher Guaranteed Delivery

Guaranteed delivery is controlled by the Connector.kafkaProducerGuaranteeDelivery configuration property. When Connector.kafkaProducerGuaranteeDelivery=true, CDC Publisher overrides other configuration settings such as the following settings, as needed:

```
Connector.sendMaintainOrder=true
Connector.kafkaProducerSendSynchronously=true
Connector.connectorCheckpointAll=true
```

CDC Publisher overrides or adds multiple CDC Publisher and Kafka-related properties to help track the success of each message that is sent. To determine which CDC Publisher properties were changed, look for CDCPUB_15050 messages in the CDC Publisher log. For example:

```
[CDCPUB_15050] The process [Connector InputQueue Handler] is changing the property
[kafkaProducerSendSynchronously] with the value [false] to the new value [true] to support
Guaranteed Delivery.
```

Also, when Connector.kafkaProducerGuaranteeDelivery=true, CDC Publisher overrides the following Kafka producer property values:

- acks
- batch.size
- linger.ms
- retries

To determine which producer property settings were added or altered, look for messages CDCPUB_15025 and CDCPUB_15026 in the CDC Publisher log. For example:

```
[CDCPUB_15025] The process [Connector Sender Helper 1] is adding the Kafka property [acks] with
the value [all] to support the Guaranteed Delivery configuration setting.

[CDCPUB_15026] The process [Connector Sender Helper 1] is changing the Kafka property
[batch.size] with the value [1000000] to the new value [0] to support Guaranteed Delivery.
```

While guaranteed delivery is enabled, no other performance tuning methods in CDC Publisher have a significant impact.

If the consumer applications can tolerate duplicate records and you want to improve performance, you can disable guaranteed delivery. To disable CDC Publisher guaranteed delivery, set the Connector.kafkaProducerGuaranteeDelivery property to false:

```
Connector.kafkaProducerGuaranteeDelivery=false
```

Effects of Disabling CDC Publisher Guaranteed Delivery

CDC Publisher guaranteed delivery provides protection against sending duplicate messages to the target. After you disable guaranteed delivery and restart CDC Publisher, duplicate messages are more likely to be sent to the target. If your consumer applications cannot tolerate duplicate records, do not disable guaranteed delivery.

Enabling CDC Publisher Asynchronous Communication with Kafka

Versions of the Kafka client API earlier than version 1.0 allow messages to be sent to Kafka in synchronous mode or asynchronous mode.

Synchronous communication lessens the number of messages that might be duplicated when the CDC Publisher restarts. Asynchronous communication provides better performance by allowing messages to be sent to Kafka without waiting for acknowledgement that the prior messages were received.

To enable asynchronous communication, set the Connector.kafkaProducerSendSynchronously property to false:

```
Connector.kafkaProducerSendSynchronously=false
```

To find the current Connector.kafkaProducerSendSynchronously setting, look for CDCPUB_15029 messages in the CDC Publisher log. For example:

```
[CDCPUB_10059] The configuration property [Connector.kafkaProducerSendSynchronously] has the
value [false]
```

CDC Publisher Leveraging of Kafka Partitioning

Kafka topics can be partitioned. Partitioning allows for improved Kafka and CDC Publisher broker performance. However, the order in which messages are written to Kafka topic partitions might not be the same order in which they are distributed to consumers. By default, the CDC Publisher sends all messages to the same partition in a topic to maintain the message order. You can configure CDC Publisher to send messages to separate partitions to improve performance.

Managing Partitioning

To send messages to multiple partitions in a Kafka topic but not maintain the message order, set the following configuration property:

```
Connector.sendMaintainOrder=false
```

To find the current CDC Publisher setting, look for message CDCPUB_15025 and the sendMaintainOrder setting in the CDC Publisher log:

```
[CDCPUB_10059] The configuration property [Connector.sendMaintainOrder] has the value [false]
```

Partitioning Effects

- When Kafka topics use partitioning, CDC Publisher performance can improve because partitioning allows Kafka to write to multiple partitions in parallel.
- Messages consumed from Kafka topics that have multiple partitions might not be in the same order in which they were written to the topics.

Reducing Checkpointing I/O

By default, the CDC Publisher writes a checkpoint to the local checkpoint file each time a message that contains a change is written to Kakfa. This behavior is controlled by the CDC Publisher Connector.connectorCheckpointAll property. The default value is true. You can set this property to false to allow checkpointing to occur less frequently, which improves the CDC Publisher performance.

Note: Checkpoint frequency be modified only when guaranteed delivery is disabled by specifying Connector.kafkaProducerGuaranteeDelivery=false in the cdcPublisherKafka.cfg configuration file. When guaranteed delivery is enabled and you attempt to change the Connector.connectorCheckpointAl setting, CDC Publisher sets Connector.connectorCheckpointAl to true and reports the change in the following message:

```
INFO [CDCPUB_15050] The process [Connector InputQueue Handler] is changing the property
[connectorCheckpointAll] with the value [false] to the new value [true] to support Guaranteed
Delivery.
```

When you restart CDC Publisher, it begins sending messages to Kafka from the last checkpoint that was written to the checkpoint file. If a checkpoint was not written for each message, the saved checkpoint at restart might not represent the last message sent to Kafka, which can result in duplicate messages.

In CDC Publisher versions earlier than 1.3, all checkpoints are written to a local file. Beginning with CDC Publisher 1.3, you can specify the Connector.checkpointsInTarget property to write checkpoint information to Kafka as part of the Kafka message header.

Changing File-Based Checkpoint Storage

You control file-based checkpoint storage by setting the following CDC Publisher configuration properties:

```
Connector.connectorCheckpointAll=false
Connector.connectorCheckpointCommits=false
Connector.checkpointMessageFrequency=0
Connector.checkpointTimeFrequency=0
```

Set Connector.connectorCheckpointAll=false to disable I/O to a local file for writing a checkpoint for every message sent to Kafka. After connectorCheckpointAll is disabled, you can use the remaining three configuration properties to enable periodic checkpoints. The connectorCheckpointCommits, checkpointMessageFrequency, and checkpointTimeFrequency properties can be used in any combination. A common configuration might be to take a checkpoint on both a message frequency and a time frequency, to provide protection during busy and idle time periods.

Changing Target-Based Checkpoint Storage

Set Connector.checkpointsInTarget=true to enable storing checkpoint information in Kafka. To control target-based checkpoint storage, set the following CDC Publisher configuration properties:

```
Connector.checkpointsInTarget=true
Connector.connectorCheckpointAll=false
Connector.connectorCheckpointCommits=false
Connector.checkpointMessageFrequency=0
Connector.checkpointTimeFrequency=0
```

Also adjust the other checkpoint configuration settings to create a file-based backup of checkpoint information, which you can use to restore target-based checkpoint information if the Kafka topics are purged or deleted.

Checkpointing Considerations

- The number of messages re-sent to Kafka when CDC Publisher restarts depends on the accuracy of the last checkpoint found.
- For checkpoints that are written to a local file, I/O reduction can improve CDC Publisher performance. However, after a CDC restart, duplicate messages can occur.
- For checkpoints written to the target, local I/O reduction can improve CDC Publisher performance. Also the risk of duplicate messages after a restart is lower because all checkpoints are written to the target. However, CDC Publisher restart might be delayed while topics are examined to determine the correct checkpoint from which to start.

To find the current CDC Publisher setting, look for message CDCPUB_15025 in the CDC Publisher log for the checkpoint.

Increasing the Size of the CDC Publisher Internal Pending Buffer

The CDC Publisher internal pending buffer contains references to the records that are sent to Kafka. References are held in the buffer until CDC Publisher receives an acknowledgment from Kafka that the record was successfully written to Kafka. If the internal pending buffer is full, CDC Publisher stops sending new messages to Kafka until acknowledgements are received and the associated references can be removed from the internal buffer. When no space is available in the buffer, no messages are sent to Kafka.

Changing the Internal Buffer Size

You can specify the CDC Publisher Connector.connectorPendingBufferSize property in the cdcPublisherKafka.cfg configuration file to set the maximum number of items that the buffer can hold. Initially, Informatica recommends that you set this property to 25 and determine if the increase in pending buffer size is providing the improvement you need.

Determining If the Buffer Size Is Adequate

To determine if the size of the buffer is adequate, periodically gather internal statistics from CDC by running the PwxCDCInfo STATS=INTERNAL command while the CDC Publisher is under load. And then search the report output for statistics. For example:

```
Connector Connector OutputQueue Handler PendingQueueSize   0
Connector Connector OutputQueue Handler WaitingOnFullPendingQueue true
```

- If the PendingQueueSize value is typically much lower than the Connector.connectorPendingBufferSize setting, the size is adequate.
- If the PendingQueueSize value is typically equal to the value specified by the Connector.connectorPendingBufferSize property, the buffer is frequently full, which might delay sending new events to Kafka.
- If the WaitingOnFullPendingQueue value is consistently true, the queue buffer is waiting for a response from Kafka before sending more data.

Note: A large pending queue buffer might negatively impact performance. Do not arbitrarily specify a large Connector.connectorPendingBufferSize value.

Buffer Size Considerations

Review the following consideration related to the internal pending buffer size:

- An excessively large CDC Publisher pending queue requires more resources to maintain.
- An undersized CDC Publisher pending queue creates delays when sending messages to Kafka.
- The following relationships exist between the CDC Publisher pending queue size and Kafka producer properties:
  - The Kafka acks property must exist in the producer properties file and be equal to 1 or all. If the acks property is missing or set to 0, CDC Publisher receives acknowledgments from Kafka slowly or misses some acknowledgments, which delays clearing the pending queue.
  - The Kafka batch.size property and the CDC Publisher pending queue size affect each other.
    The Kafka batch.size property is measured in bytes. The CDC Publisher connectorPendingBufferSize property is measured in number of messages. After the size limit in one of these properties is met, adjusting the other property is less useful.

    For example, after the CDC Publisher connectorPendingBufferSize limit is reached, increasing the size to the Kafka batch.size property might not have an impact if the CDC Publisher has stopped sending messages. The Kafka batch.size threshold will never be met. Conversely, if the Kafka batch.size threshold is reached first, adding more capacity to the pending queue might not show an additional benefit.
  - The Kafka linger.ms producer property controls how long the Kafka producer waits before closing a batch of messages that has not yet exceeded the size set in the Kafka batch.size property. A linger.ms value that is missing or set to 0 might not allow the Kafka producer batch to fill, whereas a large linger.ms value might create additional latency.

  When setting the Kafka linger.ms and batch.size producer properties, try starting with linger.ms=1 and batch.size=512000. Then adjust the CDC Publisher connectorPendingBufferSize value to fit. Remember to check connectorPendingBufferSize when adjusting Kafka producer properties.

Increasing the Size of Selected CDC Publisher Internal Queues

The CDC Publisher is multithreaded. Data flows internally between CDC Publisher threads buffered in internal memory queues. These queues have small queue sizes by default, which accommodate the CDC Publisher default configuration that uses guaranteed delivery.

If you disable guaranteed delivery to improve performance, you can also increase the size of these internal memory queues to increase parallelism between the CDC Publisher threads and to further improve performance.

Managing the Size of Selected CDC Publisher Internal Queues

You can manage the sizes of multiple types of queues individually by setting internal CDC Publisher properties. Typically, the queue sizes have similar values, starting at 1000 for tuning purposes. For example:

```
Extract.extractorOutputQueueSize=1000
Extract.extractorConsumerInputStreamSize=1000 [introduced in CDC Publisher 1.3]
Extract.extractorRawEventBufferSize=1000
Formatter.formatterOutputQueueSize=1000
Formatter.formatterRawEventBufferSize=1000
Connector.connectorRawEventBufferSize=1000
```

Measuring the Size of the Queues

To determine if the queue sizes are reaching the configured limits, periodically gather internal statistics from CDC Publisher. Run the PwxCDCInfo STATS=INTERNAL command while the CDC Publisher is under load. Then search for "QueueSize" in the statistics output to find the current queue sizes. On Linux, you can use the grep command.

A CDC Publisher pending queue that is full can degrade performance. Messages stop flowing while the queue is full. Conversely, internal memory queues with a low volume of messages or no messages might indicate no work needs to be done or a failure occurred in the threads writing to these queues, which caused them to not keep up with the threads reading from the queues.

Preferably, the internal memory queues are often full or nearly full.

Impact of Queue Size

Internal memory queue size depends on the number of messages from the source and on the size of the messages. Large messages consume more memory than small messages. A queue size based on the number of messages might consume more memory than a queue based on the amount of data from the source.

Undersized queues reduces parallelism between the internal CDC Publisher threads, which degrades overall performance.

Hardware

CDC Publisher is multithreaded. CDC Publisher performance improves significantly if you increase the number of cores or amount of memory on the machine where you run CDC Publisher. For the best performance, use a machine with at least 16 cores and at least 32 GB of memory.

Monitoring Performance

Informatica offers a shell script, called getstats, for measuring record throughput over time.

The Knowledge Base article *Scripts and Utilities for Monitoring PowerExchange CDCPublisher*, which is available at https://kb.informatica.com/howto/6/Pages/23/608853.aspx, provides a link to the getstats script and describes how to use the script with the PwxCDCInfo utility.

Use the getstats script to periodically query CDC Publisher internal statistics. The script uses the grep command to search the OutputRecordsWritten count and calculates the approximate number of records per second at each interval set in the script.

Author

Julie Henry

Acknowledgements

The author would like to acknowledge the PowerExchange CDC Publisher development team for their help with this article.